Práctica 5

Pruebas unitarias en Django

Pruebas de la API para verificar la creación, consulta, actualización y eliminación de categorías y productos.

Archivo probado: `products/tests.py`
Autenticación: cada prueba usa un usuario de prueba con `force_authenticate` porque los endpoints requieren JWT.
Resultado: 6 pruebas ejecutadas correctamente.

Prueba 1

Crear una categoría

> Envía una petición POST al endpoint de categorías con el nombre “Ropa”. Comprueba que la API responda con código 201, que existan dos categorías y que la nueva categoría se haya guardado.

Fragmento de products/tests.py

```python
def test_create_category(self):
    response = self.client.post('/api/categories/', {'name': 'Ropa'})

    self.assertEqual(response.status_code, 201)
    self.assertEqual(Category.objects.count(), 2)
    self.assertTrue(Category.objects.filter(name='Ropa').exists())
```

Qué valida: creación correcta y persistencia de la categoría.

Prueba 2

Listar categorías

> Solicita la lista de categorías mediante GET. Comprueba que la respuesta sea exitosa, que contenga una categoría y que incluya “Electronica”.

Fragmento de products/tests.py

```python
def test_list_categories(self):
    response = self.client.get('/api/categories/')

    self.assertEqual(response.status_code, 200)
    self.assertEqual(len(response.data), 1)
    self.assertEqual(response.data[0]['name'], 'Electronica')
```

Qué valida: consulta de categorías y contenido de la respuesta.

Prueba 3

Crear un producto

> Envía los datos obligatorios de un producto mediante POST. Comprueba que la API responda con código 201, que aumente el total de productos y que el producto “Mouse” exista.

Fragmento de products/tests.py

```python
def test_create_product(self):
    response = self.client.post('/api/products/', {
        'name': 'Mouse',
        'description': 'Mouse inalambrico',
        'price': 500.00,
    })

    self.assertEqual(response.status_code, 201)
    self.assertEqual(Product.objects.count(), 2)
    self.assertTrue(Product.objects.filter(name='Mouse').exists())
```

Qué valida: creación del producto y guardado en la base de datos.

Prueba 4

Listar productos

> Realiza una petición GET al endpoint de productos. Comprueba que la respuesta sea exitosa, que devuelva un producto y que el nombre recibido sea “Laptop”.

Fragmento de products/tests.py

```python
def test_list_products(self):
    response = self.client.get('/api/products/')

    self.assertEqual(response.status_code, 200)
    self.assertEqual(len(response.data), 1)
    self.assertEqual(response.data[0]['name'], 'Laptop')
```

Qué valida: listado de productos y datos principales de la respuesta.

Prueba 5

Actualizar un producto

> Actualiza el producto existente usando PUT. Comprueba que la API responda con código 200 y que el nombre y el precio nuevos queden guardados.

Fragmento de products/tests.py

```python
def test_update_product(self):
    response = self.client.put(
        f'/api/products/{self.product.id}/',
        {
            'name': 'Laptop Pro',
            'description': 'Laptop HP actualizada',
            'price': 18000.00,
        },
    )

    self.assertEqual(response.status_code, 200)
    self.product.refresh_from_db()
    self.assertEqual(self.product.name, 'Laptop Pro')
    self.assertEqual(self.product.price, 18000.00)
```

Qué valida: actualización de datos y persistencia después de recargar el registro.

Prueba 6

Eliminar un producto

> Elimina el producto mediante DELETE. Comprueba que la API responda con código 204 y que el registro ya no exista en la base de datos.

Fragmento de products/tests.py

```python
def test_delete_product(self):
    response = self.client.delete(
        f'/api/products/{self.product.id}/',
    )

    self.assertEqual(response.status_code, 204)
    self.assertFalse(Product.objects.filter(id=self.product.id).exists())
```

Qué valida: eliminación correcta y ausencia del producto eliminado.

Resultado de la ejecución

python manage.py test

Salida obtenida en la terminal

```
Found 6 test(s).
Creating test database for alias 'default'...
System check identified some issues:

......
----------------------------------------------------------------------
Ran 6 tests in 2.198s

OK
Destroying test database for alias 'default'...
```

Las seis pruebas unitarias fueron ejecutadas correctamente.

Nota: Django mostró advertencias informativas sobre DEFAULT_AUTO_FIELD, pero no hubo fallos en las pruebas.